

UVA 7: Diccionarios

Guía de ejercicios

Enfatizar que la principal utilidad de los diccionarios es relacionar llaves con valores en una colección de datos. A partir de una llave, recuperamos el valor asociado utilizando []. Esta debe ser la operación principal. La iteración se hace sobre las llaves. Los diccionarios no tienen orden. Se puede consultar la existencia de llaves utilizando in.

Primera parte

1. Final de la Copa América

```
Chile = {
    1: 'Claudio Bravo',
    4: 'Mauricio Isla',
    17: 'Gary Medel',
    18: 'Gonzalo Jaro',
    15: 'Jean Beausejour',
    8: 'Arturo Vidal',
    21: 'Marcelo Díaz',
    20: 'Charles Aranguiz',
    6: 'José Pedro Fuenzalida',
    7: 'Alexis Sánchez',
    11: 'Eduardo Vargas'
}
```

```
Argentina = {
    1: 'Sergio Romero',
    4: 'Gabriel Mercado',
    17: 'Nicolás Otamendi',
    13: 'Ramiro Funes Mori',
    16: 'Marcos Rojo',
    6: 'Lucas Biglia',
    14: 'Javier Mascherano',
    19: 'Éver Banega',
    10: 'Lionel Messi',
    9: 'Gonzalo Higuaín',
    7: 'Ángel Di María'
}
```

- (a) Recuperar el jugador de un equipo con un número particular
- (b) ¿Qué pasa si no está?
- (c) Mostrar en pantalla la alineación de ambos equipos: “Con el número x, fulano”
- (d) Encontrar el número de un jugador particular de un equipo. Destacar la diferencia cuando se quiere buscar un valor, con respecto a cuando se recupera a partir de la llave.
- (e) Mostrar las parejas de jugadores que usan el mismo número

2. Fórmula 1

Los valores asociados con las llaves pueden ser también colecciones, como listas, e incluso otros diccionarios.

```
F1 = {
    'Mercedes': ['Lewis Hamilton', 'Valtteri Bottas'],
    'Ferrari': ['Sebastian Vettel', 'Charles Leclerc'],
    'Red Bull Racing': ['Max Verstappen', 'Alexander Albon'],
    'McLaren': ['Carlos Sainz', 'Lando Norris'],
    'Renault': ['Daniel Ricciardo', 'Esteban Ocon'],
    'AlphaTauri': ['Pierre Gasly', 'Daniil Kvyat'],
    'Racing Point': ['Sergio Pérez', 'Lance Stroll'],
    'Alfa Romeo Racing': ['Kimi Räikkönen', 'Antonio Giovinazzi'],
    'Haas F1 Team': ['Romain Grosjean', 'Kevin Magnussen'],
    'Williams': ['George Russell', 'Nicholas Latifi']
}
```

- (a) Imprimir los integrantes de un equipo particular.
- (b) ¿Cómo saber a qué equipo pertenece un piloto en particular?

3. Frutas y verduras

Se tiene un diccionario que asocia productos con una lista que contiene el precio y el stock.

```
verdulería = {
    'Brócoli': [900, 10],
    'Pimentón': [800, 5],
    'Limones': [1500, 0],
    'Lechuga': [700, 10],
    'Palta': [3800, 7],
    'Tomates': [1200, 20],
    'Pepino': [500, 0],
    'Zanahorias': [700, 12],
    'Zapallo italiano': [450, 8],
    'Papas': [950, 25],
    'Frutillas': [3400, 2],
    'Peras': [1500, 0],
    'Manzanas': [1600, 4],
    'Naranjas': [1800, 12],
    'Plátanos': [1100, 3],
    'Kiwis': [2800, 0],
    'Mandarinas': [2200, 4]
}
```

- (a) Determinar el precio de un producto en particular
- (b) Agregar condición para validar existencia
- (c) Determinar el stock de un producto en particular, indicando si no existe
- (d) Determinar el producto más caro de la verdulería

(e) Suponga una lista de *strings* con las compras que se quieren hacer, por ejemplo:

```
compras = ['Manzanas', 'Mangos', 'Papas', 'Tomates', 'Pepino', 'Pimentón', 'Plátanos']
```

Se había escrito un programa que procesaba la lista de compras, indicando también si algún producto no estaba disponible en la verdulería, ya sea porque no lo venden o porque no hay stock actualmente. Al finalizar imprimía una lista con las compras efectuadas y el monto total a pagar. Reconstruya el código a continuación para que haga lo indicado.

```
print('No hay', elemento)
for elemento in compras:
    total = 0
    print(bolsa)
    else:
        lista = verdulería[elemento]
        if lista[1] > 0:
            print(total)
            lista[1] -= 1
            if elemento in verdulería:
                total += lista[0]
            else:
                bolsa = []
                print('Negocio no vende', elemento)
                bolsa.append(elemento)
```

Solución:

```
total = 0
bolsa = []
for elemento in compras:
    if elemento in verdulería:
        lista = verdulería[elemento]
        if lista[1] > 0:
            bolsa.append(elemento)
            total += lista[0]
            lista[1] -= 1
        else:
            print('No hay', elemento)
    else:
        print('Negocio no vende', elemento)
print(bolsa)
print(total)
```

4. Números en diferentes idiomas

Como se dijo antes, los valores asociados a las llaves pueden ser colecciones, incluso diccionarios como en este caso.

```
numeros = {
    'Francés': {1: 'un', 2: 'deux', 3: 'trois', 4: 'quatre', 5: 'cinq',
                6: 'six', 7: 'sept', 8: 'huit', 9: 'neuf', 10: 'dix'},
    'Portugués': {1: 'um', 2: 'dois', 3: 'três', 4: 'quatro', 5: 'cinco',
                  6: 'seis', 7: 'sete', 8: 'oito', 9: 'nove', 10: 'dez'},
    'Inglés': {1: 'one', 2: 'two', 3: 'three', 4: 'four', 5: 'five',
               6: 'six', 7: 'seven', 8: 'eight', 9: 'nine', 10: 'ten'},
    'Español': {1: 'uno', 2: 'dos', 3: 'tres', 4: 'cuatro', 5: 'cinco',
                6: 'seis', 7: 'siete', 8: 'ocho', 9: 'nueve', 10: 'diez'},
    'Italiano': {1: 'uno', 2: 'due', 3: 'tre', 4: 'quattro', 5: 'cinque',
                 6: 'sei', 7: 'sette', 8: 'otto', 9: 'nove', 10: 'dieci'},
    'Alemán': {1: 'eins', 2: 'zwei', 3: 'drei', 4: 'vier', 5: 'fünf',
               6: 'sechs', 7: 'sieben', 8: 'acht', 9: 'neun', 10: 'zehn'},
    'Mapudungún': {1: 'kiñe', 2: 'epu', 3: 'küla', 4: 'meli', 5: 'kechu',
                   6: 'kayu', 7: 'regle', 8: 'pura', 9: 'aylla', 10: 'mari'}
}
```

- (a) Determinar cómo se dice un número particular en un idioma en particular
- (b) Imprimir cómo se dice un número en todos los idiomas registrados
- (c) ¿Se puede determinar en qué idioma está un número en particular? ¿Qué cosas hay que tener en cuenta? (R/ Los valores se pueden repetir, por ej., 'uno' puede ser Español o Italiano)

5. Planetas

Estructura de las listas que aparecen como valores en el diccionario:

```
[ distancia_al_sol_en_kms, diámetro_ecuatorial_en_kms,
  período_orbital_en_años_Tierra, período_de_rotación_en_días_Tierra,
  masa_relativa_a_Tierra ]

planetas = {
    'Mercurio': [57910000, 4880, 0.241, 58.6, 0.06],
    'Venus': [108200000, 12000, 0.72, 243, 0.82],
    'Tierra': [149600000, 12756, 1, 1, 1],
    'Marte': [227940000, 6794, 1.52, 1.03, 0.11],
    'Júpiter': [778833000, 142984, 5.20, 0.414, 318],
    'Saturno': [1429400000, 120536, 9.55, 0.426, 95],
    'Urano': [2870990000, 51118, 19.22, 0.718, 14.16],
    'Neptuno': [4504300000, 49492, 30.66, 0.6745, 17.2]
}
```

- (a) Imprimir los planetas en orden descendente de diámetro, es decir, del más grande al más pequeño.

Solución

```
lista = []
for planeta in planetas:
    diametro, _, _, _ = planetas[planeta]
    lista.append([diametro, planeta])
lista.sort()
lista.reverse()
for _, planeta in lista:
    print(planeta)
```

(b) Determinar el planeta con el menor período de rotación.

6. Sinónimos

Se cuenta con un diccionario que asocia palabras con sinónimos

```
sinonimos = {
    'abundante': 'mucho',
    'acabar': 'terminar',
    # ...
}
```

(a) Escribir un programa que transforme un texto reemplazando las palabras que tienen sinónimo y dejando sin cambios las que no tienen. Se puede suponer que no hay signos de puntuación y que las palabras se separan con un único espacio en blanco, excepto la última que no tiene espacio al final.

Para concentrarse en el manejo de los diccionarios, entregar a los estudiantes el esqueleto del programa que hace la descomposición en palabras del texto:

```
texto += ' '
palabra = ''
i = 0
while i < len(texto):
    if texto[i] == ' ':
        print(palabra) # Reemplazar esto con el procesamiento de la palabra
        palabra = ''
    else:
        palabra += texto[i]
    i += 1
```

Solución:

```

texto = 'En la penumbra vio su silueta con anteojos y un bonito
cabello y sintió miedo y ganas de volver a su casa'
texto += ' '
modificado = ''
palabra = ''
i = 0
while i < len(texto):
    if texto[i] == ' ':
        if palabra in sinonimos:
            modificado += (sinonimos[palabra]+' ')
        else:
            modificado += (palabra+' ')
            palabra = ''
    else:
        palabra += texto[i]
    i += 1
print(modificado)

```

(b) Construir un nuevo diccionario que incluya también los sinónimos en el orden inverso, por ejemplo, si en el diccionario original esta:

'abundante': 'mucho'

en el nuevo diccionario se debe incluir:

'mucho': 'abundante',

7. Patrón de contar

Considere un programa que lee como entrada los nombres de una gran cantidad de personas, hasta que se ingresa la palabra 'FIN' en lugar del nombre. Complete lo faltante para que, utilizando un diccionario, cuente la cantidad de veces que aparece cada nombre distinto.

```

contar_nombres = {}
nombre = input('Nombre: ')
while nombre != 'FIN':
    if nombre in contar_nombres:
        contar_nombres[nombre] = 0
    contar_nombres[nombre] += 1

    nombre = input('Nombre: ')
print(contar_nombres)

```

8. Patrón de clasificar

Se tiene una lista de listas que almacenan los números de RUT y el monto de una compra realizada durante el día en un almacén. Un cliente puede comprar varias veces. Se tiene además una función que convierte una lista de listas en un diccionario.

```
def transformar(lista):
    d={}
    for rut, monto in lista:
        if rut not in d:
            d[rut]=0
        d[rut] = d[rut] + monto
    return d

montos = [
    ['17123123-8',100], ['16323232-0',200], ['17123123-8',50],
    ['18788998-1',150] #...
]
resultado = transforma(montos)
```

Preguntas:

1. resultado['16323232-0'] tiene el valor: ____

R/ 200

2. resultado['17123123-8'] tiene el valor: ____

R/ 150

3. resultado['17001230-5'] tiene el valor: ____

R/ Error

4. len(resultado) tiene el valor: ____

R/ 3

5. '17123123-8' in x evalúa como: ____

R/ True

Segunda parte: Estos ejercicios se enfocan en los patrones para contar y clasificar utilizando diccionarios.

Se tienen los datos de los vendedores en un diccionario llamado `personas`, el cual tienen como llave el código del vendedor y como valor el nombre del vendedor:

```
personas = {12:"Jin", 29:"Suga", 7:"Jimin"}
```

Además se tiene una lista de listas, donde cada lista está compuesta por el código del vendedor y el monto total de la venta.

```
ventas = [
    [7,345230],[12,446550],[7,987230],[29,762470],
    [7,263680],[29,673640] #, ...
]
```

La organización requiere saber el NOMBRE del vendedor que logró un monto mayor de venta. A continuación se presenta un programa que resuelve este problema, pero usted lo debe completar para que funcione correctamente.

```
def obtener_ventas(ventas):
    d = _____
    for cod, monto in ventas:
        if cod _____ d:
            _____ = 0
            d[cod] += monto
    return d

personas = {12:"Jin", 29:"Suga", 7:"Jimin"}
ventas = [
    [7,345230],[12,446550],[7,987230],[29,762470],
    [7,263680],[29,673640] #, ...
]

dic = obtener_ventas(ventas)
nombre_mejor = personas[12]
cantidad_mejor = dic[12]

for cod in dic:
    if _____ > cantidad_mejor:
        nombre_mejor = personas[cod]
        cantidad_mejor = _____

print("Nombre mejor vendedor(a):", nombre_mejor)
```